



254383 Algorithm Design and Analysis

อ.พิเศษพงศ์ สุธาพันธ์

ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





Week 6

Transform-and-Conquer

อ.พิเศษพงศ์ สุธาพันธ์

ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





Transform-and-Conquer

วัตถุประสงค์

1. รู้วิธีการทำงานของเทคนิค

Transform-and-Conquer

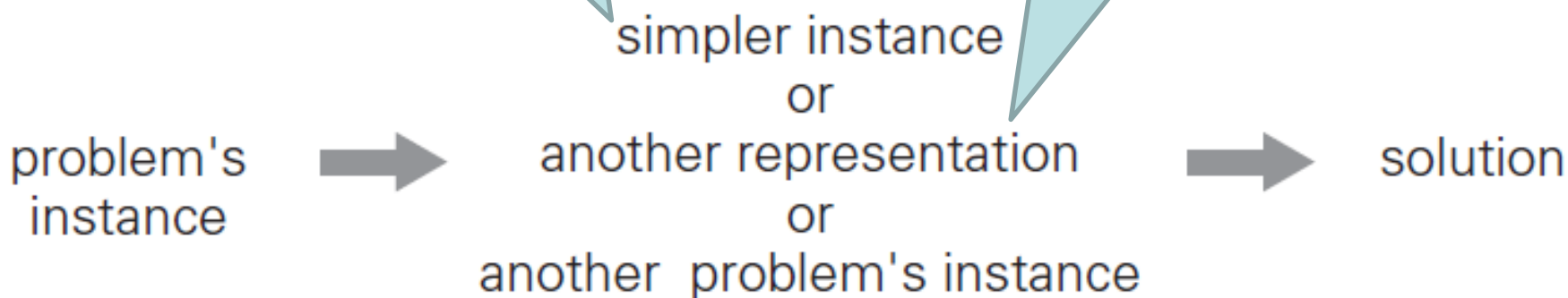
2. สามารถนำไปประยุกต์ใช้งานได้



Transform and conquer

จัดทำข้อมูลให้ง่ายต่อการนำไปใช้

นำเสนอในรูปแบบอื่นแทน



เปลี่ยนเป็นรูปแบบปัญหาอื่น



Transform-and-Conquer

มี 3 แนวคิดหลักต่อไปนี้

- Instance simplification
- Representation change
- Problem reduction



Transform and conquer

Flavours of Transform and Conquer

1. Instance Simplification = a more convenient instance of the same problem

- Presorting
- Gaussian elimination

2. Representation Change = a different representation of the same instance

- Balanced search trees
- Heaps and heapsort
- Polynomial evaluation by Horner's rule

3. Problem Reduction = a different problem altogether

- Reductions to graph problems



Transform-and-Conquer

Instance simplification

เปลี่ยนให้อยู่ในรูปแบบที่ง่ายขึ้น เช่น
การเปลี่ยนรูปแบบ input ในกรณีที่
ข้อมูลไม่ได้จัดเรียง ก็จัดเรียงก่อนแล้ว
ค่อยมาแก้ปัญหา



Transform-and-Conquer

Instance simplification

ก่อน

26 33 39 18 26 47 32 40 24 36

หลัง

18 24 26 26 32 33 36 39 40 47



Transform-and-Conquer

Representation change

เปลี่ยน input ให้เป็นรูปแบบใหม่ ที่จะ
ทำให้แก้ปัญหาได้ง่ายขึ้น เช่น การ
เปลี่ยน input ไปเป็น tree



Transform-and-Conquer

Representation change

เปลี่ยน input ให้เป็นรูปแบบใหม่ ที่จะ
ทำให้แก้ปัญหาได้ง่ายขึ้น เช่น การ
เปลี่ยน input ไปเป็น tree



Transform-and-Conquer

Problem reduction

เปลี่ยนปัญหาให้เป็นปัญหาใหม่ ที่เป็น
ปัญหาที่มีการแก้ไขได้แล้ว เช่น การ
เปลี่ยนเป็น graph



Transform-and-Conquer

6.1 Presorting



Transform-and-Conquer

ต.ย. 1

ตรวจสอบค่าซ้ำกันใน array

ALGORITHM *PresortElementUniqueness*($A[0..n - 1]$)

//Solves the element uniqueness problem by sorting the array first

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Returns “true” if A has no equal elements, “false” otherwise
sort the array A

for $i \leftarrow 0$ **to** $n - 2$ **do**

if $A[i] = A[i + 1]$ **return false**

return true



Transform-and-Conquer

ประสิทธิภาพ

$$T(n) = T_{\text{sort}}(n) + T_{\text{scan}}(n) \in \Theta(n \log n) + \Theta(n) = \Theta(n \log n)$$



Transform-and-Conquer

ต.ย. 2

หาตัวเลขที่ซ้ำกันมากที่สุด (Mode)

การหาโดยทั่วไปก็เอาแต่ละตัวไปเทียบกับตัวที่เหลือ ซึ่ง จะเป็น $n * (n-1)$





Transform-and-Conquer

ต.ย. 2

หาตัวเลขที่ซ้ำกันมากที่สุด (Mode)

ประสิทธิภาพคือ

$$C(n) = \sum_{i=1}^n (i - 1) = 0 + 1 + \cdots + (n - 1) = \frac{(n - 1)n}{2} \in \Theta(n^2)$$



Transform-and-Conquer

ต.ย. 2

หาตัวเลขที่ซ้ำกันมากที่สุด (Mode)

ALGORITHM *PresortMode*($A[0..n - 1]$)

//Computes the mode of an array by sorting it first

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: The array's mode

sort the array A

$i \leftarrow 0$ //current run begins at position i

$modefrequency \leftarrow 0$ //highest frequency seen so far

while $i \leq n - 1$ **do**

$runlength \leftarrow 1$; $runvalue \leftarrow A[i]$

while $i + runlength \leq n - 1$ **and** $A[i + runlength] = runvalue$

$runlength \leftarrow runlength + 1$

if $runlength > modefrequency$

$modefrequency \leftarrow runlength$; $modevalue \leftarrow runvalue$

$i \leftarrow i + runlength$

return $modevalue$



Transform-and-Conquer

ต.ย. 2

หาตัวเลขที่ซ้ำกันมากที่สุด (Mode)

ประสิทธิภาพคือ

$$T(n) = T_{\text{sort}}(n) + T_{\text{scan}}(n) \in \Theta(n \log n) + \Theta(n) = \Theta(n \log n)$$



Transform-and-Conquer

ต.ย. 3

Searching problem

ประสิทธิภาพคือ

$$T(n) = T_{\text{sort}}(n) + T_{\text{search}}(n) = \Theta(n \log n) + \Theta(\log n) = \Theta(n \log n)$$



Transform-and-Conquer

6.2 Gaussian Elimination



Transform-and-Conquer

จากสมการ

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2$$

$\vdots$

$$a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n$$

ต้องการอยู่ในรูปนี้

$$a'_{11}x_1 + a'_{12}x_2 + \cdots + a'_{1n}x_n = b'_1$$

$$a'_{22}x_2 + \cdots + a'_{2n}x_n = b'_2$$

$\vdots$

$$a'_{nn}x_n = b'_n.$$



Transform-and-Conquer

แสดงในรูป

$$Ax = b \implies A'x = b'$$

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}, b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$A' = \begin{bmatrix} a'_{11} & a'_{12} & \dots & a'_{1n} \\ 0 & a'_{22} & \dots & a'_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a'_{nn} \end{bmatrix}, b' = \begin{bmatrix} b'_1 \\ b'_2 \\ \vdots \\ b'_n \end{bmatrix}$$

โดยที่



ตัวอย่างที่ 2 ข้อมูลที่ให้มา

Linear equation

ต้องใช้

Gaussian Elimination

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n\end{aligned}$$



$$\begin{aligned}a'_{11}x_1 + a'_{12}x_2 + \cdots + a'_{1n}x_n &= b'_1 \\a'_{22}x_2 + \cdots + a'_{2n}x_n &= b'_2 \\&\vdots \\a'_{nn}x_n &= b'_n.\end{aligned}$$

ทำงาน 2 stages:

- forward elimination
- backward substitution



Transform-and-Conquer

Elementary Operations

จาก matrix A ให้อยู่ในรูป upper-triangular coefficient matrix A'

1. Exchanging two equations of the system
2. Replacing an equation with its nonzero multiple
3. Replacing an equation with a sum or difference of this equation and some multiple of another equation



Transform-and-Conquer

ต.ย. 1

$$2x_1 - x_2 + x_3 = 1$$

$$4x_1 + x_2 - x_3 = 5$$

$$x_1 + x_2 + x_3 = 0$$

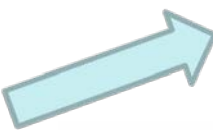


$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{array}{l} \\ \text{row 2} - \frac{4}{2} \text{ row 1} \\ \text{row 3} - \frac{1}{2} \text{ row 1} \end{array}$$



$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix} \begin{array}{l} \\ \\ \text{row 3} - \frac{1}{2} \text{ row 2} \end{array}$$

Forward Elimination


$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & 0 & 2 & -2 \end{bmatrix}$$



Transform-and-Conquer

ต.ย. 1

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & 0 & 2 & -2 \end{bmatrix}$$

แก้สมการโดย **back substitutions**

$$x_3 = (-2)/2 = -1, \quad x_2 = (3 - (-3)x_3)/3 = 0.$$

$$\text{and } x_1 = (1 - x_3 - (-1)x_2)/2 = 1$$



Transform-and-Conquer

ALGORITHM *ForwardElimination*($A[1..n, 1..n]$, $b[1..n]$)

//Applies Gaussian elimination to matrix A of a system's coefficients,
//augmented with vector b of the system's right-hand side values

//Input: Matrix $A[1..n, 1..n]$ and column-vector $b[1..n]$

//Output: An equivalent upper-triangular matrix in place of A with the
//corresponding right-hand side values in the $(n + 1)$ st column

for $i \leftarrow 1$ **to** n **do** $A[i, n + 1] \leftarrow b[i]$ //augments the matrix

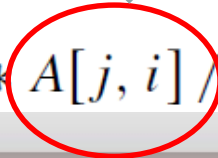
for $i \leftarrow 1$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** n **do**

for $k \leftarrow i$ **to** $n + 1$ **do**

$A[j, k] \leftarrow A[j, k] - A[i, k] * A[j, i] / A[i, i]$

มีปัญหาดังตรงตัวแปรนี้





Transform-and-Conquer

ALGORITHM *ForwardElimination*($A[1..n, 1..n]$, $b[1..n]$)

//Applies Gaussian elimination to matrix A of a system's coefficients,
//augmented with vector b of the system's right-hand side values

//Input: Matrix $A[1..n, 1..n]$ and column-vector $b[1..n]$

//Output: An equivalent upper-triangular matrix in place of A with the
//corresponding right-hand side values in the $(n + 1)$ st column

for $i \leftarrow 1$ **to** n **do** $A[i, n + 1] \leftarrow b[i]$ //augments the matrix

for $i \leftarrow 1$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** n **do** Temp_x = $a[j, i]$

for $k \leftarrow i$ **to** $n + 1$ **do**

$A[j, k] \leftarrow A[j, k] - A[i, k] * A[j, i] / A[i, i]$

$\in \Theta(n^3)$

Temp_x





Transform-and-Conquer

ALGORITHM *BetterForwardElimination*($A[1..n, 1..n]$, $b[1..n]$)

//Implements Gaussian elimination with partial pivoting

//Input: Matrix $A[1..n, 1..n]$ and column-vector $b[1..n]$

//Output: An equivalent upper-triangular matrix in place of A and the
//corresponding right-hand side values in place of the $(n + 1)$ st column

for $i \leftarrow 1$ **to** n **do** $A[i, n + 1] \leftarrow b[i]$ //appends b to A as the last column

for $i \leftarrow 1$ **to** $n - 1$ **do**

$pivotrow \leftarrow i$

for $j \leftarrow i + 1$ **to** n **do**

if $|A[j, i]| > |A[pivotrow, i]|$ $pivotrow \leftarrow j$

for $k \leftarrow i$ **to** $n + 1$ **do**

$swap(A[i, k], A[pivotrow, k])$

for $j \leftarrow i + 1$ **to** n **do**

$temp \leftarrow A[j, i] / A[i, i]$

for $k \leftarrow i$ **to** $n + 1$ **do**

$A[j, k] \leftarrow A[j, k] - A[i, k] * temp$

$\in \Theta(n^3)$



Transform-and-Conquer

```
algorithm BackwardSubst( $A[1..n, 1..n + 1]$ )  
  // Second stage to solve  $Ax = b$  by Gaussian Elim.  
  // Input: An upper-triangular matrix  $A$   
  // Output: Solution column-vector  $x$   
  for  $i \leftarrow n$  downto 1 do  
     $s \leftarrow A[i, n + 1]$   
    for  $j \leftarrow i + 1$  to  $n$  do  
       $s \leftarrow s - A[i, j] * x[j]$   
     $x[i] \leftarrow s / A[i, i]$   
return  $x$ 
```

วิเคราะห์ประสิทธิภาพ

- ลำดับการโตะของ Forward Elimination = $O(n^3)$
- ลำดับการโตะของ Backward Substitution = $O(n^2)$
- ประสิทธิภาพการทำงานทั้งหมด
 $= O(n^3) + O(n^2)$
 $= O(n^3)$



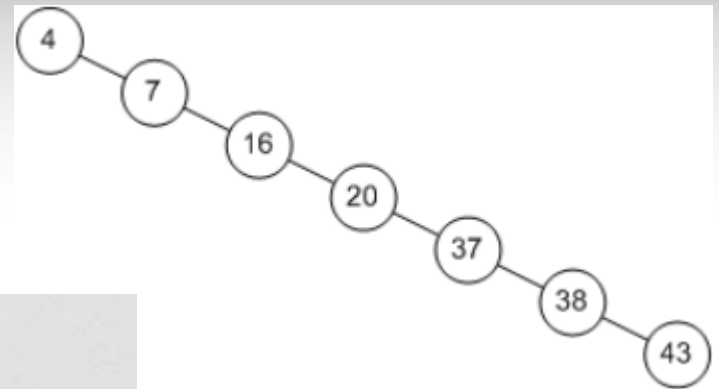
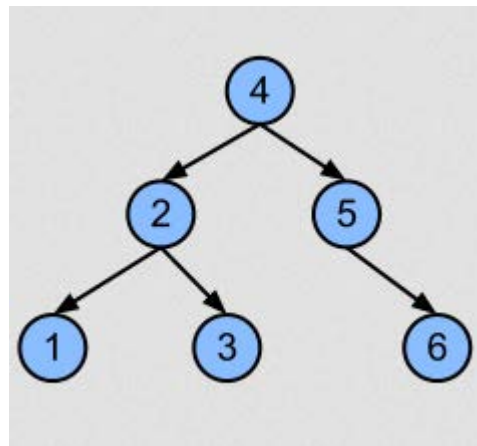
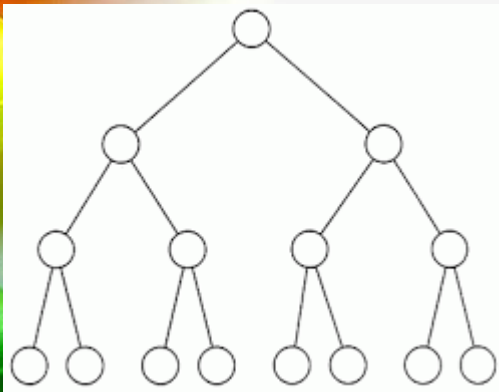
Transform-and-Conquer

6.3 Balanced Search Trees



Transform-and-Conquer

Binary Tree



ในกรณีแย่ที่สุดในการค้นหาใน Binary Tree คือ $\Theta(n)$



Transform-and-Conquer

ในกรณีแย่ที่สุดในการค้นหาใน Binary Tree คือ $\Theta(n)$

โดย tree จะมีความสูงเป็น $n-1$ ซึ่งเป็น
unbalanced binary tree

ซึ่งต้องมีการปรับให้เป็น balanced binary tree

มีแนวคิด 2 แบบ คือ



Transform-and-Conquer

1. AVL tree

เป็นการปรับต้นไม้โดยดูความสูงของต้นไม้
ย่อยทางซ้ายและขวาให้มีความต่างไม่เกิน

2. 2-3 trees

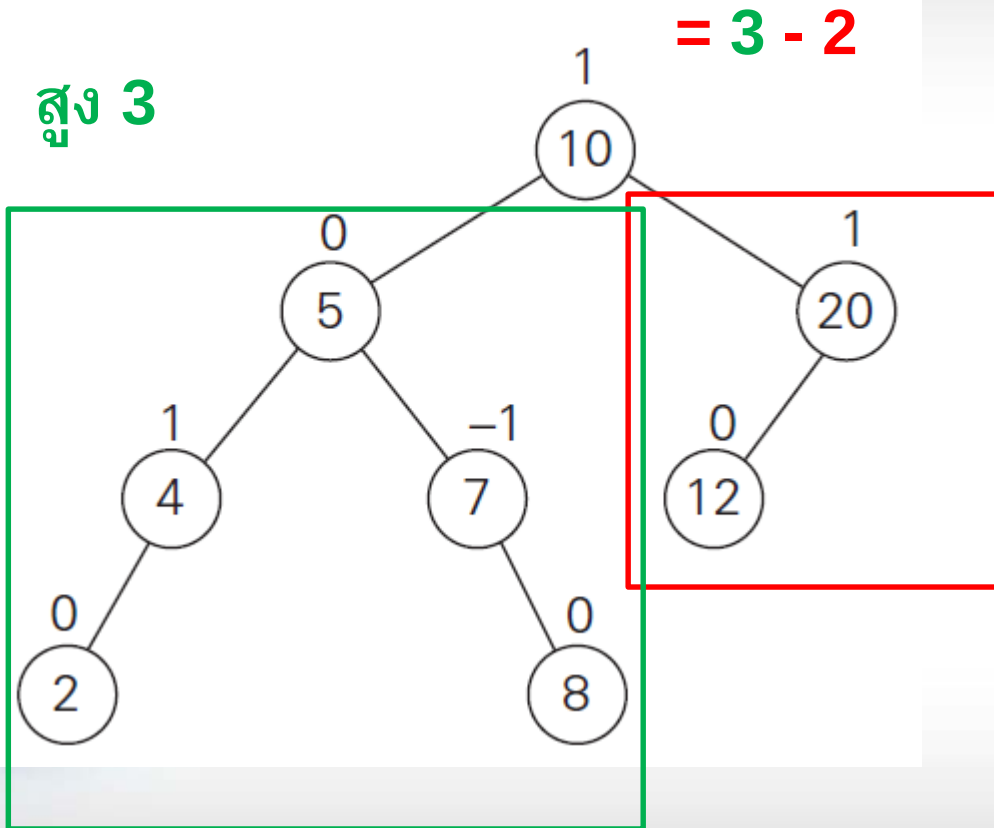
ซึ่งเป็นรูปแบบประเภท B-tree



Transform-and-Conquer

AVL tree

ស្ទូង 3

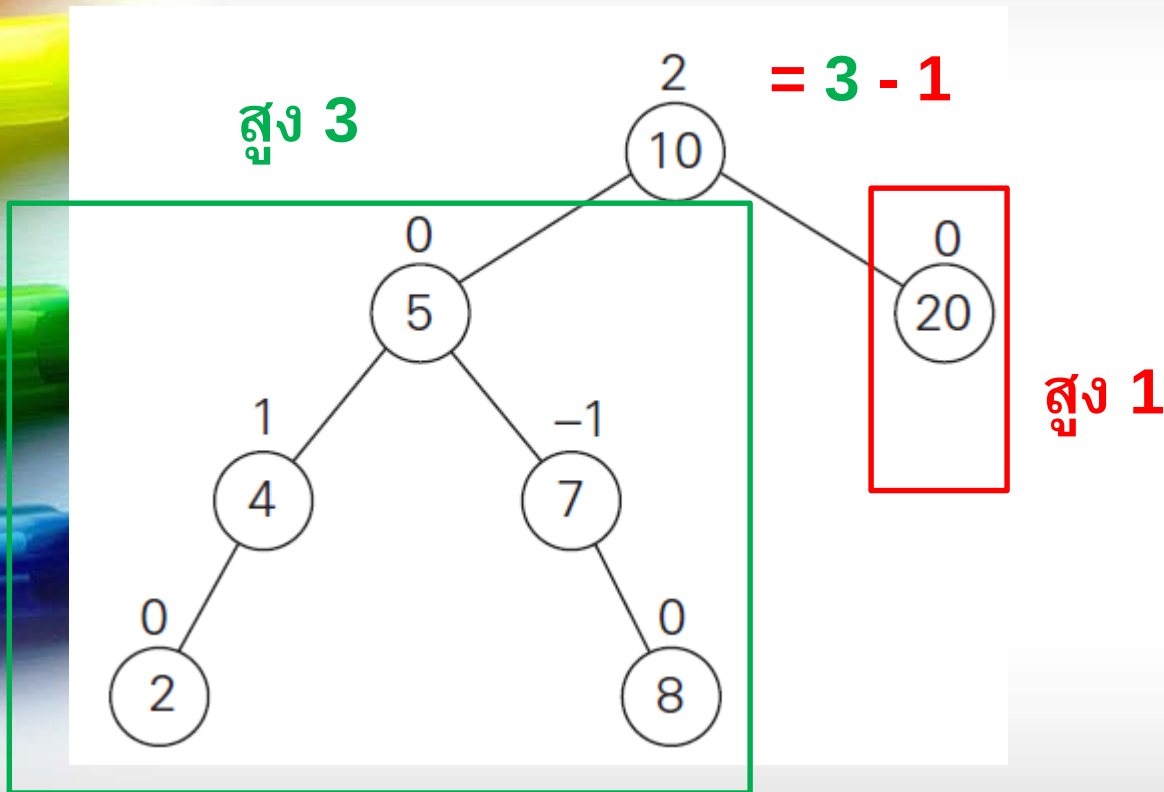


ស្ទូង 2



Transform-and-Conquer

ไม่ใช่ AVL tree

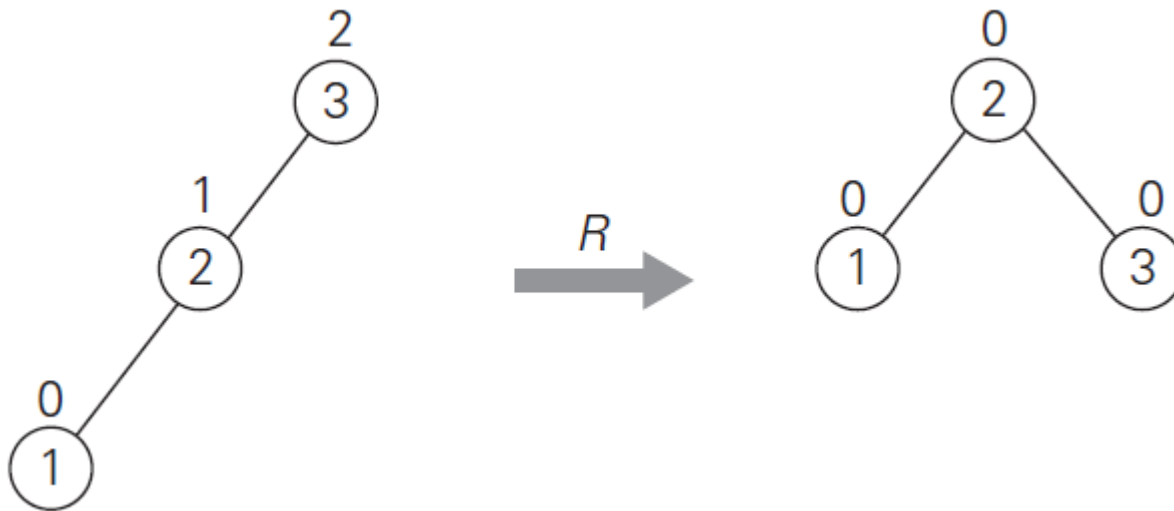




Transform-and-Conquer

การปรับให้สมดุล ของ AVL มีการหมุนอยู่ 4 แบบ

1. Single right rotation หรือ R-rotation

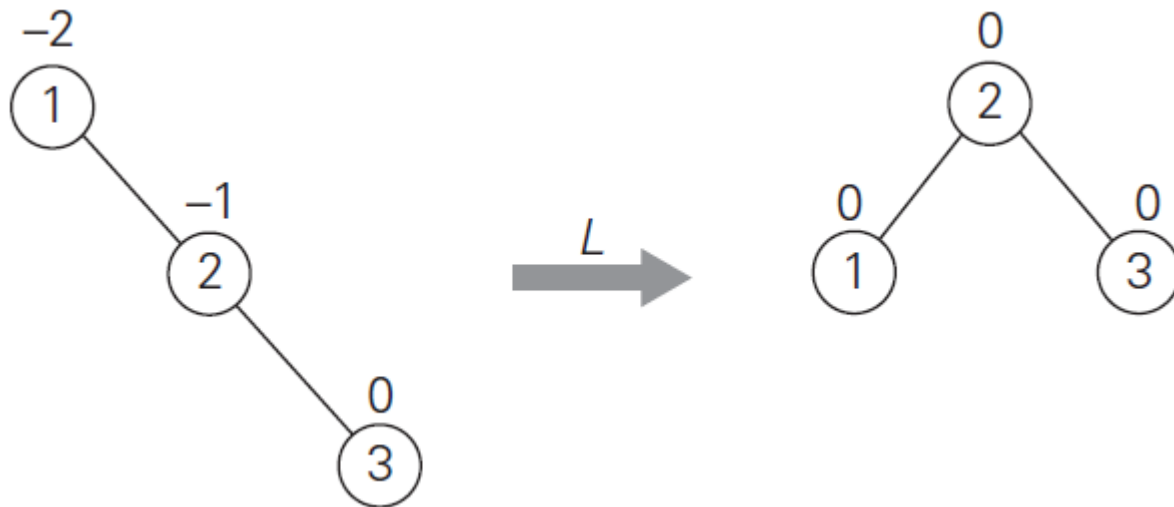




Transform-and-Conquer

การปรับให้สมดุล ของ AVL มีการหมุนอยู่ 4 แบบ

2. Single left rotation หรือ L-rotation





Transform-and-Conquer

การปรับให้สมดุล ของ AVL มีการหมุนอยู่ 4 แบบ

3. Double left-right rotation หรือ *LR-rotation*





Transform-and-Conquer

การปรับให้สมดุล ของ AVL มีการหมุนอยู่ 4 แบบ

4. Double right-left rotation หรือ RL-rotation

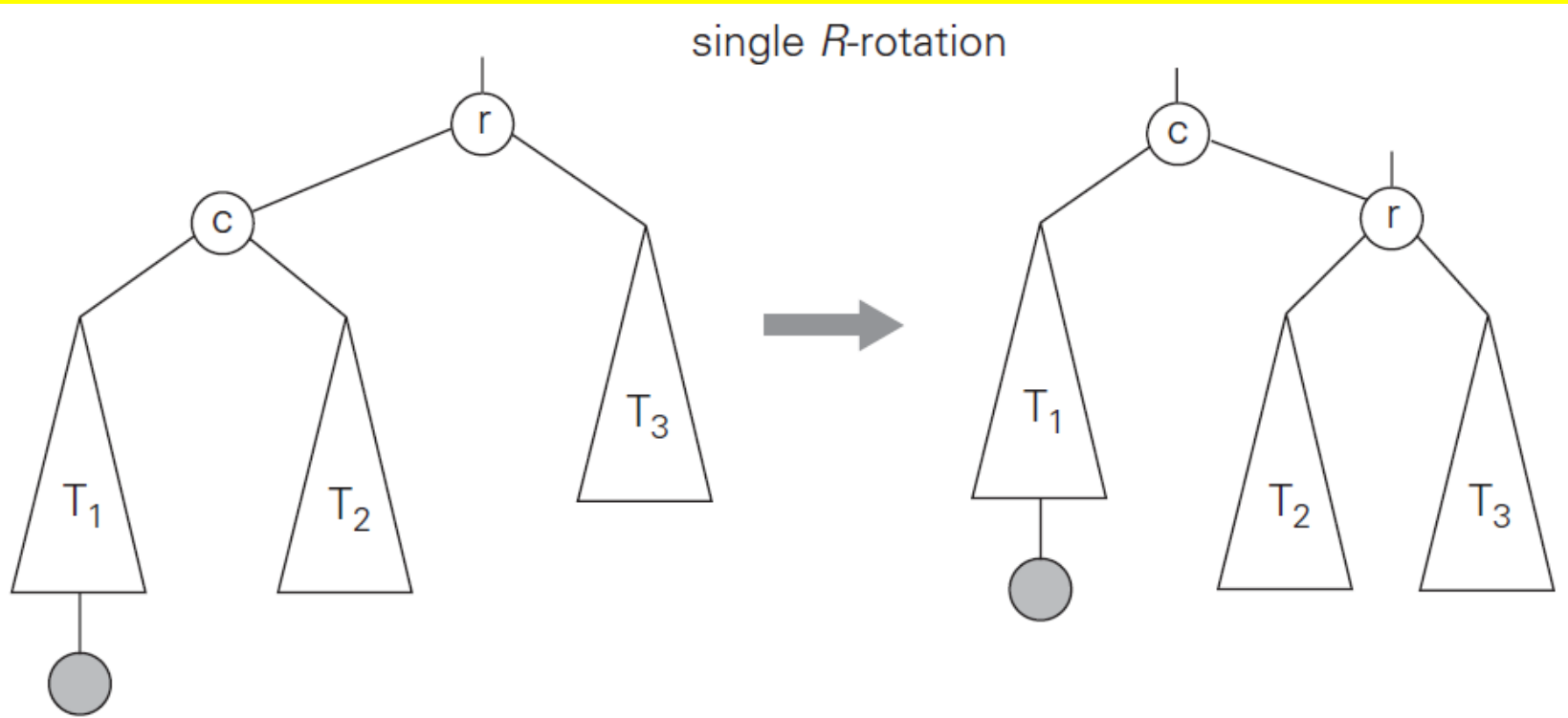




Transform-and-Conquer

การปรับให้สมดุล ของ AVL มีการหมุนอยู่ 4 แบบ

ตัวอย่างรูปทั่วไปในการปรับเมื่อมีการเพิ่มโหนด

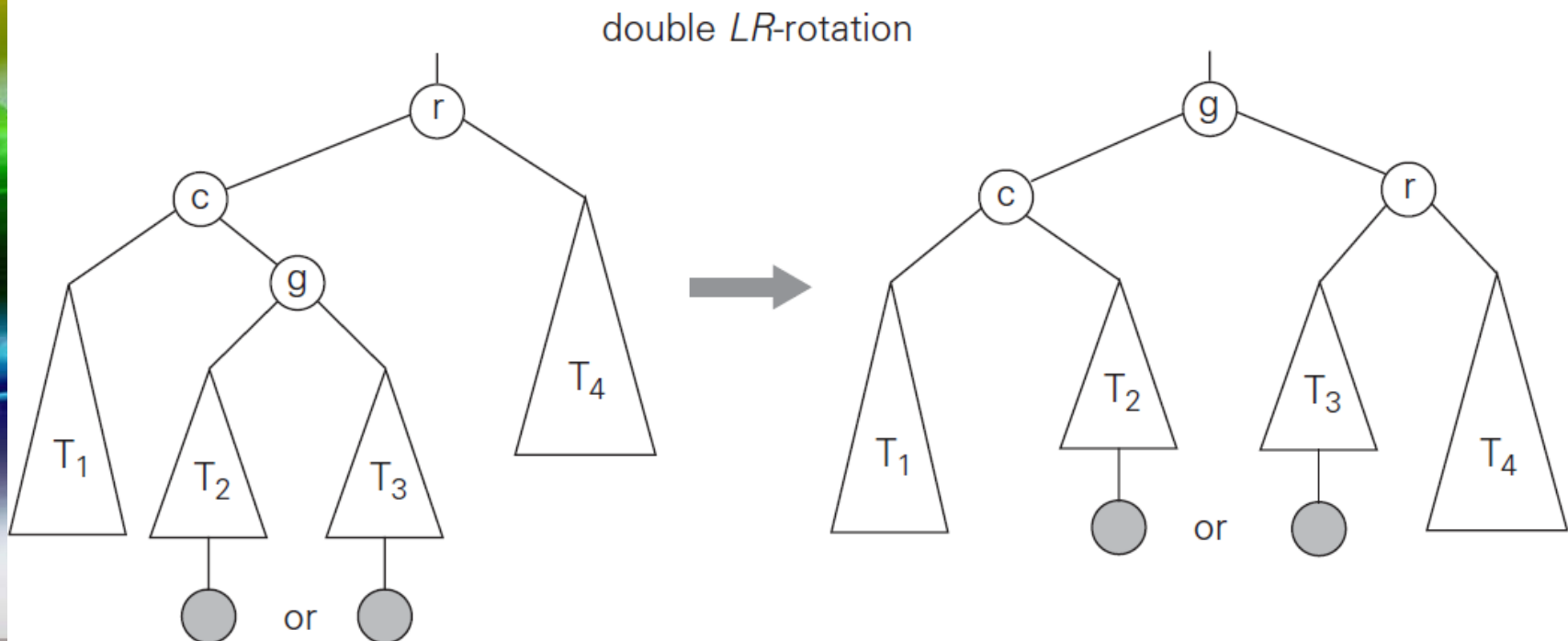




Transform-and-Conquer

การปรับให้สมดุล ของ AVL มีการหมุนอยู่ 4 แบบ

ตัวอย่างรูปทั่วไปในการปรับเมื่อมีการเพิ่มโหนด

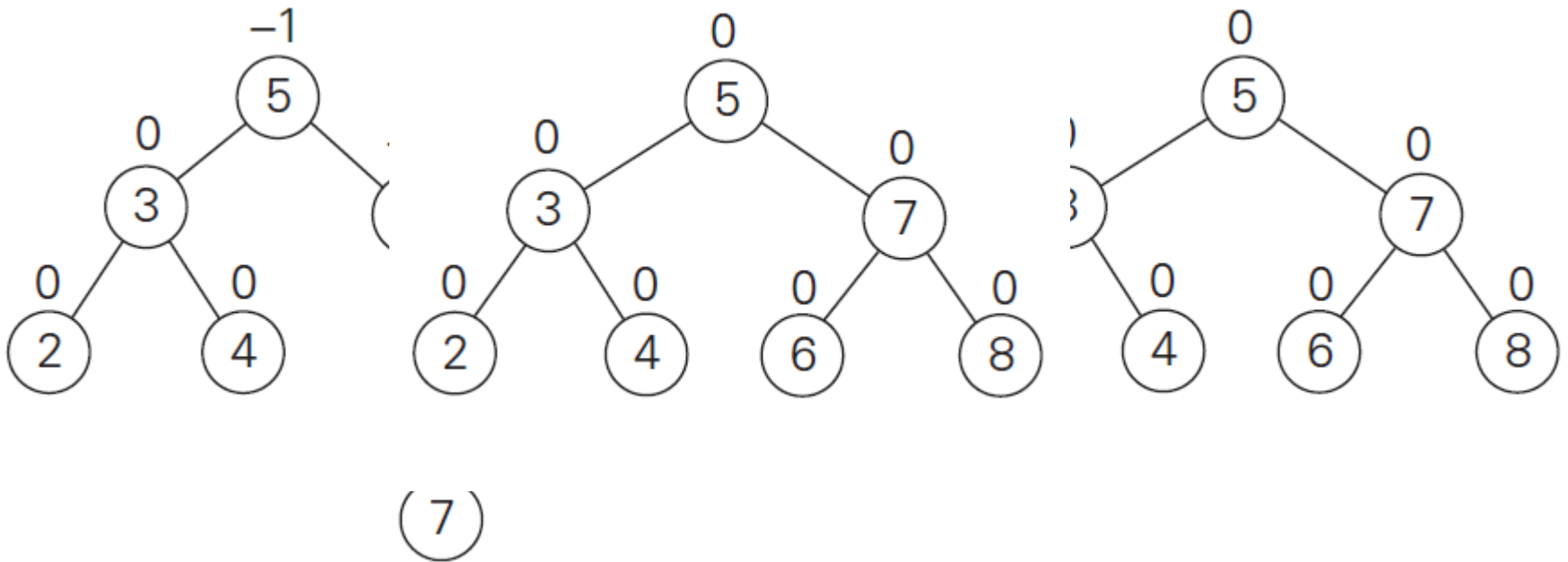




Transform-and-Conquer

ตัวอย่างการสร้าง AVL tree

5, 6, 8, 3, 2, 4, 7





Transform-and-Conquer

จงสร้าง AVL tree

1. 34 13 36 3 43 31 40 26 8 44 30 26 20 23 45

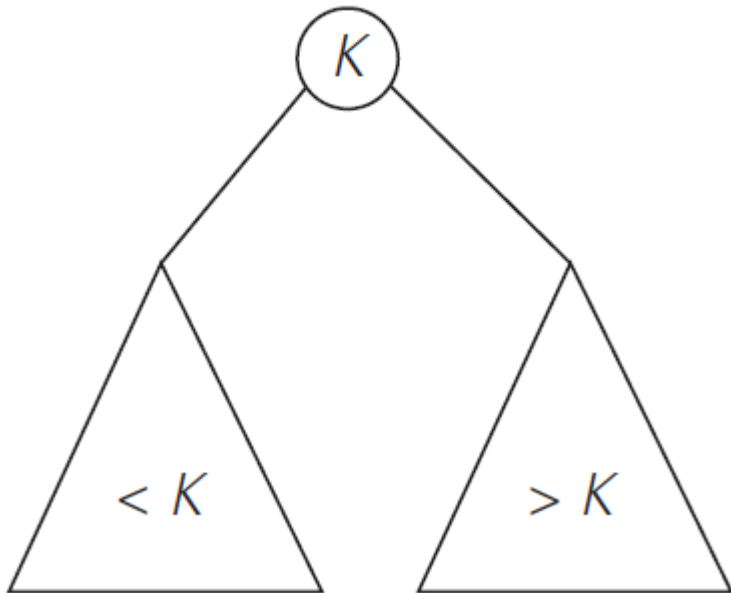
2. 25 32 14 35 19 48 42 26 10 49 16 43 41 9 45



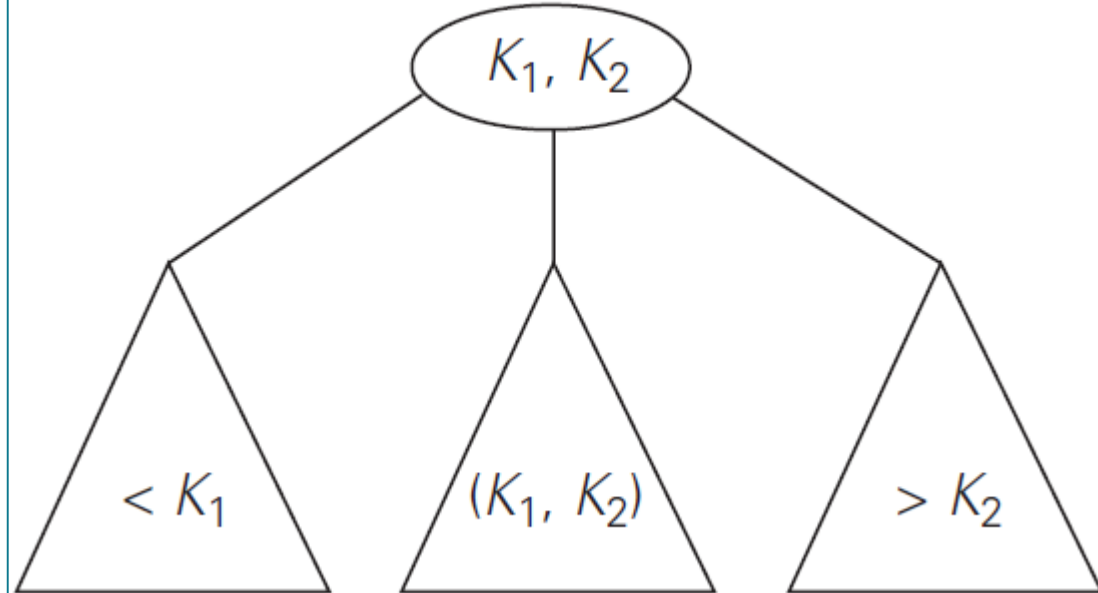
Transform-and-Conquer

รูปแบบ ของ 2-3 tree

2-node



3-node

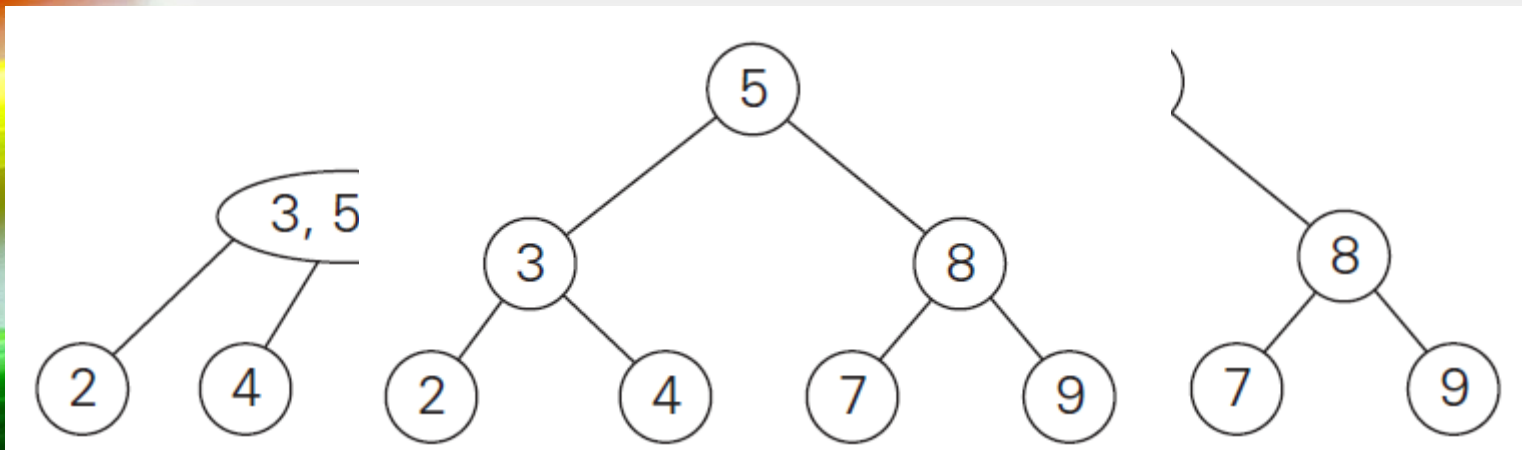




Transform-and-Conquer

การสร้างโดยวิธี 2-3 tree

9, 5, 8, 3, 2, 4, 7





Transform-and-Conquer

จงสร้างโดยวิธี 2-3 tree

12 5 27 44 6 39 26 21 41 19 34 32 38 20 37



Transform-and-Conquer

6.4 Heap and Heap Sort



Transform-and-Conquer

Heap

เป็น binary tree ที่มีเงื่อนไขดังนี้

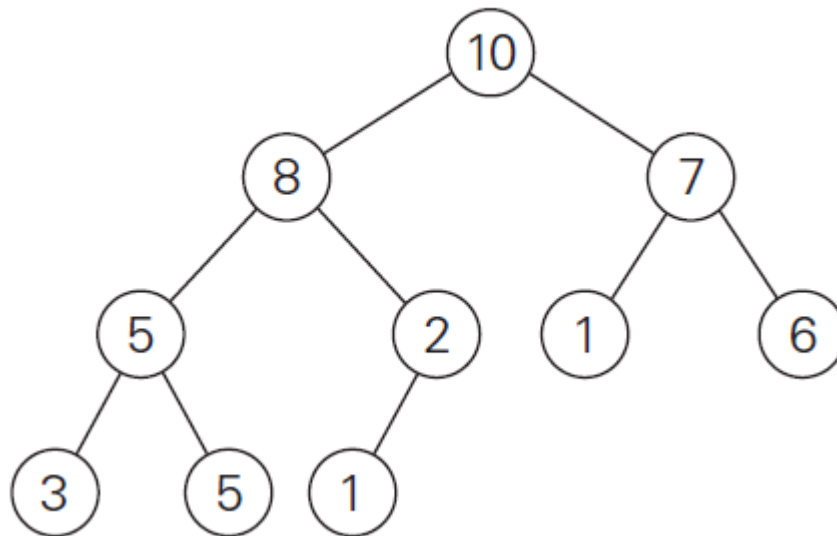
1. Shape property คือต้องมีรูปร่างเป็น complete binary
2. Parental dominance หรือ heap property คือค่าในแต่ละโหนดต้องมากกว่าหรือเท่ากับค่าโหนดลูกของตัวเอง



Transform-and-Conquer

Heap

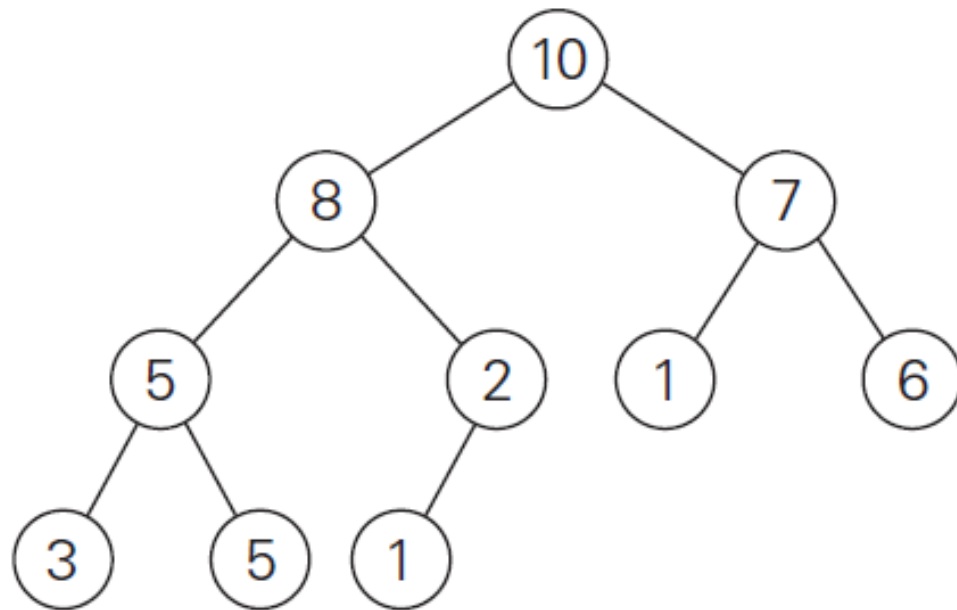
ตัวอย่าง Heap





Transform-and-Conquer

Heap



the array representation

index	0	1	2	3	4	5	6	7	8	9	10
value		10	8	7	5	2	1	6	3	5	1
	parents						leaves				



Transform-and-Conquer

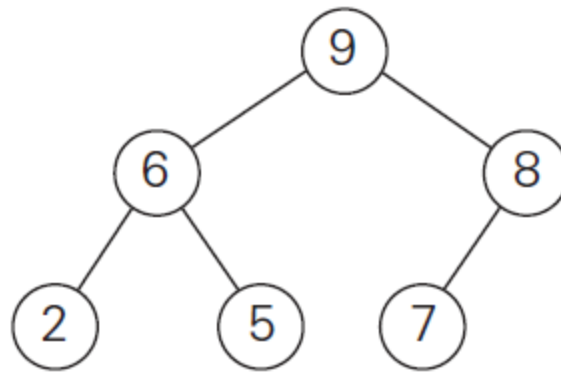
คุณสมบัติที่สำคัญของ Heap

1. ความสูงต้องเท่ากับ $\lceil \log_2 n \rceil$
2. Root จะต้องมียุคมากที่สุด
3. ทุกโหนดเรียงค่ามากไปน้อยจากบนสู่ล่าง
4. Array ที่อ้าง heap จะเรียงจากบนลงล่างซ้ายไปขวา
 - a. parent node จะถูกเก็บไว้ในตำแหน่ง $\lfloor n/2 \rfloor$ ส่วนแรกของ array และ leaf node จะเก็บตั้งแต่ตำแหน่ง $\lfloor n/2 \rfloor$
 - b. parent node อยู่ในตำแหน่ง i child node จะอยู่ในตำแหน่ง $2i$ กับ $2i+1$



Transform-and-Conquer

การสร้าง Heap





ALGORITHM *HeapBottomUp*($H[1..n]$)

//Constructs a heap from elements of a given array
// by the bottom-up algorithm

//Input: An array $H[1..n]$ of orderable items

//Output: A heap $H[1..n]$

for $i \leftarrow \lfloor n/2 \rfloor$ **downto** 1 **do**

$k \leftarrow i; \quad v \leftarrow H[k]$

$heap \leftarrow \mathbf{false}$

while not $heap$ **and** $2 * k \leq n$ **do**

$j \leftarrow 2 * k$

if $j < n$ //there are two children

if $H[j] < H[j + 1]$ $j \leftarrow j + 1$

if $v \geq H[j]$

$heap \leftarrow \mathbf{true}$

else $H[k] \leftarrow H[j]; \quad k \leftarrow j$

$H[k] \leftarrow v$



Transform-and-Conquer

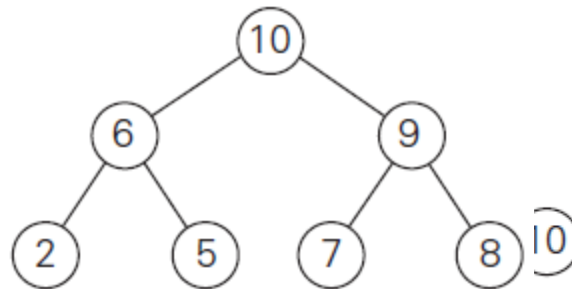
Worst Case

$$\begin{aligned}C_{worst}(n) &= \sum_{i=0}^{h-1} \sum_{\text{level } i \text{ keys}} 2(h-i) \\&= \sum_{i=0}^{h-1} 2(h-i)2^i \\&= 2(n - \log_2(n+1))\end{aligned}$$



Transform-and-Conquer

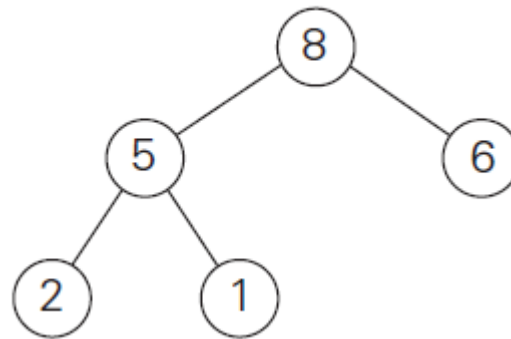
การเพิ่มข้อมูลใน heap





Transform-and-Conquer

การลบ root ออกจาก heap



Step 1 Ex Step 3 เทียบกับโหนดลูกแล้วเลื่อนลงในทางที่มากที่สุด ap

$$O(\log n)$$



Transform-and-Conquer

Heap Sort มี 2 ขั้นตอน

ขั้นตอน 1 สร้าง heap

ขั้นตอน 2 ลบ root ทำ $n-1$ ครั้ง



Transform-and-Conquer

Heap Sort มี 2 ขั้นตอน

ขั้นตอน 1 สร้าง heap

ขั้นตอน 2 ลบ root ทำ $n-1$ ครั้ง



ดูวิดีโอ heap sort



Transform-and-Conquer

2, 9, 7, 6, 5, 8

Stage 1 (heap construction)

2 9 **7** 6 5 8

2 **9** 8 6 5 7

2 9 8 6 5 7

9 **2** 8 6 5 7

9 6 8 **2** 5 7



Transf

onquer

Stage 2 (maximum deletions)

9 6 8 2 5 7

7 6 8 2 5 | **9**

8 6 7 2 5

5 6 7 2 | **8**

7 6 5 2

2 6 5 | **7**

6 2 5

5 2 | **6**

5 2

2 | **5**

2



Transform-and-Conquer

Heap Sort มีประสิทธิภาพ

$$\Theta(n \log n)$$



Transform-and-Conquer

6.5 Horner's Rule and Binary Exponentiation



Transform-and-Conquer

รูปแบบทั่วไปของ polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$



Transform-and-Conquer

Horner's Rule

เป็นการแยกตัวประกอบออกมา

$$p(x) = (\cdots (a_n x + a_{n-1})x + \cdots)x + a_0$$



Transform-and-Conquer

๓.๕.

$$p(x) = 2x^4 - x^3 + 3x^2 + x - 5$$

$$p(x) = 2x^4 - x^3 + 3x^2 + x - 5$$

$$= x(2x^3 - x^2 + 3x + 1) - 5$$

$$= x(x(2x^2 - x + 3) + 1) - 5$$

$$= x(x(x(2x - 1) + 3) + 1) - 5$$



Transform-and-Conquer

ALGORITHM *Horner*($P[0..n]$, x)

//Evaluates a polynomial at a given point by Horner's rule

//Input: An array $P[0..n]$ of coefficients of a polynomial of degree n ,

// stored from the lowest to the highest and a number x

//Output: The value of the polynomial at x

$p \leftarrow P[n]$

for $i \leftarrow n - 1$ **downto** 0 **do**

$p \leftarrow x * p + P[i]$

return p



Transform-and-Conquer

Horner's Rule

จำนวนการคูณและการบวก

$$M(n) = A(n) = \sum_{i=0}^{n-1} 1 = n$$



Transform-and-Conquer

Binary Exponentiation

รูปแบบ

$$p(x) = b_I x^I + \dots + b_i x^i + \dots + b_0$$

เช่น

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$



Transform-and-Conquer

a^n

รูปแบบ

$$a^n = a^{p(2)} = a^{b_I 2^I + \dots + b_i 2^i + \dots + b_0}$$



Transform-and-Conquer

Horner's rule for the binary polynomial $p(2)$

$p \leftarrow 1$ //the leading digit is always 1 for $n \geq 1$

for $i \leftarrow I - 1$ downto 0 do

$p \leftarrow 2p + b_i$

Implications for $a^n = a^{p(2)}$

$a^p \leftarrow a^1$

for $i \leftarrow I - 1$ downto 0 do

$a^p \leftarrow a^{2p+b_i}$

But

$$a^{2p+b_i} = a^{2p} \cdot a^{b_i} = (a^p)^2 \cdot a^{b_i} = \begin{cases} (a^p)^2 & \text{if } b_i = 0, \\ (a^p)^2 \cdot a & \text{if } b_i = 1. \end{cases}$$



Transform-and-Conquer

Left-to-right binary exponentiation method of computing a^n

ALGORITHM *LeftRightBinaryExponentiation*($a, b(n)$)

//Computes a^n by the left-to-right binary exponentiation algorithm

//Input: A number a and a list $b(n)$ of binary digits $b_I, \dots, b_0$

// in the binary expansion of a positive integer n

//Output: The value of a^n

product $\leftarrow a$

for $i \leftarrow I - 1$ **downto** 0 **do**

product $\leftarrow$ *product* * *product*

if $b_i = 1$ *product* $\leftarrow$ *product* * a

return *product*



Transform-and-Conquer

ต.ย. a^{13} เมื่อ $n = 13 = 1101_2$

binary digits of n	1	1	0	1
product accumulator	a	$a^2 \cdot a = a^3$	$(a^3)^2 = a^6$	$(a^6)^2 \cdot a = a^{13}$

$$a = 3$$

$$3 \rightarrow 3^2 \cdot 3 = 27 \rightarrow 27^2 = 729 \rightarrow 729^2 \cdot 3 = 1,594,323$$

ทำจากซ้ายไปขวา



Transform-and-Conquer

Right-to-left binary exponentiation method of computing a^n

$$a^n = a^{b_I 2^I + \dots + b_i 2^i + \dots + b_0} = a^{b_I 2^I} \cdot \dots \cdot a^{b_i 2^i} \cdot \dots \cdot a^{b_0}$$

$$a^{b_i 2^i} = \begin{cases} a^{2^i} & \text{if } b_i = 1, \\ 1 & \text{if } b_i = 0, \end{cases}$$



Transform-and-Conquer

Right-to-left binary exponentiation method of computing a^n

ALGORITHM *RightLeftBinaryExponentiation($a, b(n)$)*

//Computes a^n by the right-to-left binary exponentiation algorithm

//Input: A number a and a list $b(n)$ of binary digits $b_I, \dots, b_0$

// in the binary expansion of a nonnegative integer n

//Output: The value of a^n

$term \leftarrow a$ //initializes a^{2^i}

if $b_0 = 1$ $product \leftarrow a$

else $product \leftarrow 1$

for $i \leftarrow 1$ **to** I **do**

$term \leftarrow term * term$

if $b_i = 1$ $product \leftarrow product * term$

return $product$



Transform-and-Conquer

ต.ย. a^{13} เมื่อ $n = 13 = 1101_2$

$1 \cdot 2^3$

$1 \cdot 2^2$

$0 \cdot 2^1$

$1 \cdot 2^0$

1

1

0

1

binary digits of n

a^8

a^4

a^2

a

terms a^{2^i}

$a^5 \cdot a^8 = a^{13}$

$a \cdot a^4 = a^5$

a

product accumulator

a^{13}

a^5

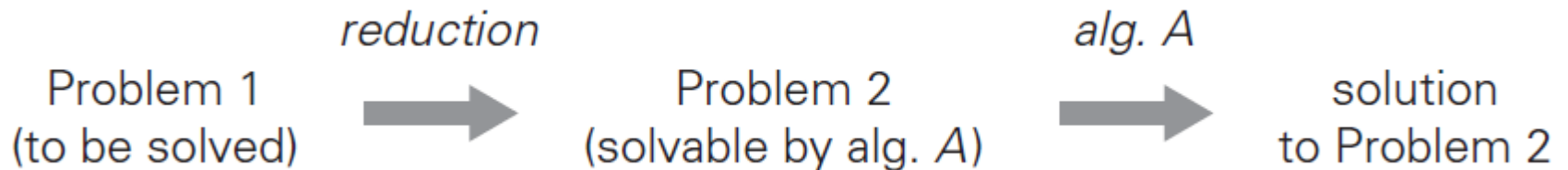
a

a



Transform-and-Conquer

6.6 Problem Reduction





Transform-and-Conquer

Computing the least common multiple

$$\text{lcm}(m, n) = \frac{m \cdot n}{\text{gcd}(m, n)},$$

แบบฝึกหัด ลองเขียนโปรแกรม lcm โดยใช้สมการนี้



Transform-and-Conquer

Computing the least common multiple

ตัวอย่าง

$$\begin{aligned}\text{LCM}(24, 60) \\ &= 1440 / 12 \\ &= 120\end{aligned}$$



Transform-and-Conquer

ปัญหา ชาวหาพาแพะ ผัก หมาป่า ข้ามแม่น้ำ
มีเงื่อนไข

1. ห้ามทิ้งแพะไว้กับหมาป่า
2. ห้ามทิ้งผักไว้กับแพะ
3. ชาวหาอุ้มสิ่งของข้ามน้ำได้ทีละอย่าง

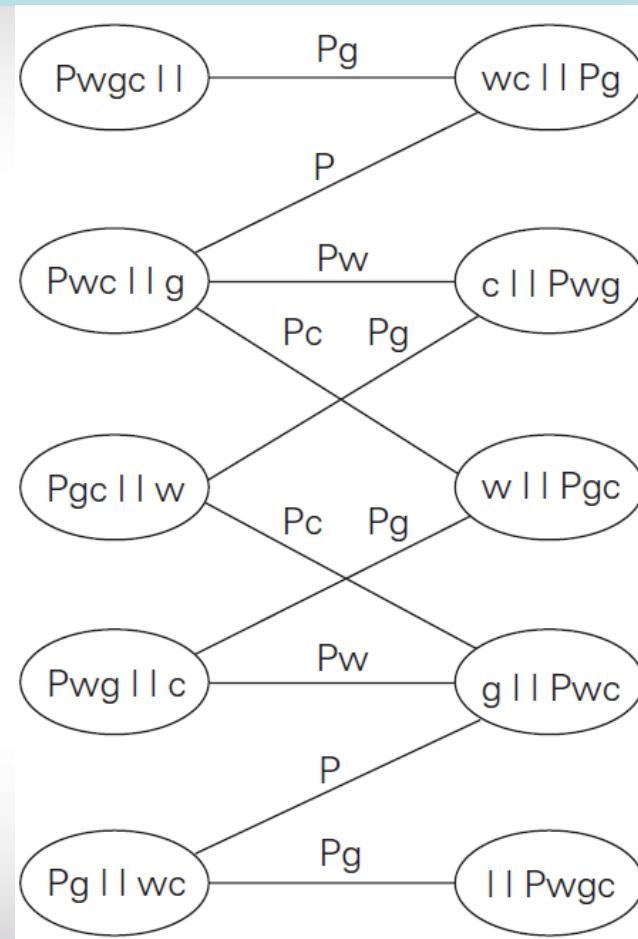


Transform-and-Conquer

Reduction to graph problems

ใช้ state-space graph ในการแก้ปัญหา
ชาวหาพาแพะ ผัก หมาป่า ข้ามแม่น้ำ

State-space graph for the peasant, wolf, goat, and cabbage puzzle





อ้างอิงและต้นฉบับ

[1] Introduction to the design & analysis of algorithm, 3rd,
Anany Levitin



Transform-and-Conquer

สรุป(ทบทวน)

1. **Presorting** คือ มีการจัดเรียงข้อมูลไว้ก่อนที่จะนำไปใช้
2. **Gaussian Elimination** คือ เป็นการแก้สมการเชิงเส้น
3. **AVL** คือ การทำ **balance tree** ปรับต้นไม้ให้มีความสูงของต้นไม้ย่อยต่างกันไม่เกิน 1
 - 3.1 สร้างโดยการหมุน
 - 3.2 สร้างโดย 2-3 tree
4. **Heap** คือ tree ที่โหนดแม่มีค่ามากกว่าโหนดลูก
5. **Horner's Rule** คือ เปลี่ยนจัดเรียงรูปแบบการคูณในฟังก์ชัน **polynomial**